NPR
College of Engineering and Technology
(An Autonomous Institution)
NBA Accredited (B.E. - CSE, ECE, EEE & Mechanical Engg.) | Accredited by NAAC with 'A' Grade
Recognized by UGC under 2 (f) | Approved by All India Council for Technical Education, New Delhi | Affiliated to Anna University, Chennai
Natham, Dindigul (Dist.) - 624 401, Tamil Nadu. | Web: www.nprcet.edu.in

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

25AI311
DATA STRUCTURES LABORATORY

MASTER LAB MANUAL

Semester III

Regulation 2025



TABLE OF CONTENTS		
S. No.	**Particulars**	**Page No.**
1	College vision and mission statement	3
2	Department Vision, Mission, POs, PEOs, PSOs	4
3	Lab syllabus	5
4	List of Experiments Mapping with COs, POs &PSOs	6
5	**University prescribed lab experiments**	
	1. Implementation of Python Fundamentals and Abstract Data Types (ADTs)	8
	2. Implementation of List ADT using Python Arrays	10
	3. Implementation of Singly, Doubly, and Circular Linked Lists	12
	4. Linked list implementations of List	16
	5. Implementation of Queue ADT and its Applications	19
	6. Implementation of Sorting and Searching Algorithms	21
	7. Implementation of Hash Tables and Collision Handling Technique	25
	8. Implementation of Tree Traversal and Binary Search Tree Operations	27
	9. Implementation of Graph Representation and Traversal Algorithms	30
	10. Implementation of Shortest Path and Minimum Spanning Tree Algorithms	32

NPR College of Engineering and Technology, Natham.

Vision

- To develop students with intellectual curiosity and technical expertise to meet the global needs.

Mission

- To achieve academic excellence by offering quality technical education using best teaching techniques.
- To improve Industry – Institute interactions and expose industrial atmosphere.
- To develop interpersonal skills along with value-based education in a dynamic learning environment.
- To explore solutions for real time problems in the society.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Vision

- To develop AI professionals of international relevance to meet the industry and societal needs with future technologies.

Mission

- To collaborate with industry and provide the state-of-the-art infrastructural facilities, with a conducive teaching-learning ambience.
- To instill in the students the knowledge for world class technical competence, entrepreneurial skill and a spirit of innovation in the area of Artificial Intelligence and Data Science to solve real world problems.
- To encourage students to pursue higher education and research.
- To inculcate right attitude and discipline and develop industry ready professionals for serving the society.

Program Educational Objectives

Graduates will be able to

1. Utilize their proficiencies in the fundamental knowledge of basic sciences, mathematics, Artificial Intelligence, data science, and statistics to build systems that require management and analysis of large volumes of data.
2. Advance their technical skills to pursue pioneering research in the field of AI and Data Science and create disruptive and sustainable solutions for the welfare of ecosystems.
3. Think logically, pursue lifelong learning and collaborate with an ethical attitude in a Multidisciplinary team
4. Design and model AI-based solutions to critical problem domains in the real world.
5. Exhibit innovative thoughts and creative ideas for an effective contribution towards the economy building.

23AI311 **DATA STRUCTURES LABORATORY** **L T P C `**
0 0 2 2

Course Objectives

- To understand and implement Abstract Data Types (ADTs) using Python
- To design and develop linear and non-linear data structures
- To apply searching, sorting, hashing, and graph algorithms in problem solving
- To develop efficient Python programs using appropriate data structures and algorithms
- To analyze algorithm performance and optimize solutions

List of Experiments

1. Implementation of Python Fundamentals and Abstract Data Types (ADTs)
2. Implementation of List ADT using Python Arrays
3. Implementation of Singly, Doubly, and Circular Linked Lists
4. Implementation of Stack ADT and its Applications
5. Implementation of Queue ADT and its Applications
6. Implementation of Searching and Sorting Algorithms
7. Implementation of Hash Tables and Collision Handling Techniques
8. Implementation of Tree Traversal and Binary Search Tree Operations
9. Implementation of Graph Representation and Traversal Algorithms
10. Implementation of Shortest Path and Minimum Spanning Tree Algorithms

Course Outcomes

The students will be able to

On completion of the course, the students will be able to		**Bloom's Level Mapped**
CO1	Implement Python programs using ADTs and recursive techniques.	Apply (K3)
CO2	Develop linear data structures such as lists, stacks, and queues.	Apply (K3)
CO3	Apply searching, sorting, and hashing techniques to solve computational problems.	Apply (K3)
CO4	Design and implement tree and graph data structures.	Analyze (K4)
CO5	Analyze and solve real-world problems using appropriate data structures and algorithms.	Analyze (K4)

List of Experiments with COs, POs & PSOs

Exp. No.	Name of the Experiment	COs	POs	PSOs
1.	Implementation of Python Fundamentals and Abstract Data Types (ADTs)	CO1	1, 2, 3, 4, 5, 10,11	1, 2 & 3
2.	Implementation of List ADT using Python Arrays	CO2	1, 2, 3, 4, 5, 9, 10, 11	1, 2 & 3
3.	Implementation of Singly, Doubly, and Circular Linked Lists	CO2	1, 2, 3, 4, 5, 9, 10, 11	1, 2 & 3
4.	Implementation of Stack ADT and its Applications	CO2	1, 2, 3, 4, 5, 9, 10, 11	1, 2 & 3
5.	Implementation of Queue ADT and its Applications	CO2	1, 2, 3, 4, 5, 9, 10, 11	1, 2 & 3
6.	Implementation of Searching and Sorting Algorithms	CO5	1, 2, 3, 4, 5, 8, 9, 10, 11	1, 2 & 3
7.	Implementation of Hash Tables and Collision Handling Techniques	CO5	1, 2, 3, 4, 5, 8, 9, 10, 11	1, 2 & 3
8.	Implementation of Tree Traversal and Binary Search Tree Operations	CO3	1, 2, 3, 4, 5, 9, 10, 11	1, 2 & 3
9.	Implementation of Graph Representation and Traversal Algorithms	CO4	1, 2, 3, 4, 5, 9, 10, 11	1, 2 & 3
10.	Implementation of Shortest Path and Minimum Spanning Tree Algorithms	CO4	1, 2, 3, 4, 5, 9, 10, 11	1, 2 & 3
				Total: 45 Periods

Program Outcomes

1. Engineering Knowledge
2. Problem Analysis
3. Design/Development of Solutions
4. Conduct Investigations of Complex Problems
5. Modern Tool Usage
6. The Engineer and Society
7. Environment and Sustainability
8. Ethics
9. Individual and Team Work
10. Communication
11. Project Management and Finance

Program Specific Outcomes

At the end of the program students will be able to

- Develop and implement AI-based processes for effective decision-making in diverse domains, including business and governance, by integrating domain-specific knowledge and advanced techniques.
- Utilize data analysis to derive actionable insights and foresights, enabling the solution of complex business and engineering problems.
- Apply theoretical knowledge of AI and Data Analytics, along with practical tools and techniques, to address societal problems, demonstrating proficiency in data analytics,visualization,and project coordination skills.

Ex. No: 01 Date:	Implementation of Python Fundamentals and Abstract Data Types (ADTs)

Aim:

To understand and implement basic Python constructs and define a simple Abstract Data Type (ADT) for a Student Record System.

Algorithm:

- Start the program and define the Student Record class with attributes roll_no, name, and an empty list marks.
- Create a Student Record object by providing the student's roll number and name.
- Add the student's marks one by one using the add_mark() method.
- Calculate the total marks by summing all the values stored in the marks list.
- Display the student's roll number, name, list of marks, and the total marks using the display_report() method.
- End the program

Program:

```
 class StudentRecord:
def __init__(self, roll_no, name):
    self.roll_no = roll_no
    self.name = name
    self.marks = []

def add_mark(self, mark):
    self.marks.append(mark)

def display_report(self):
    total = sum(self.marks)
    print(f"Roll No: {self.roll_no}")
    print(f"Name: {self.name}")
    print(f"Marks: {self.marks}")
    print(f"Total Marks: {total}")

# Testing the ADT
if __name__ == "__main__":
    student = StudentRecord(101, "Alice")
    student.add_mark(85)
    student.add_mark(90)
    student.add_mark(78)
    student.display_report()
```

Output:

```
Roll No: 101
  Name: Alice
  Marks: [85, 90, 78]
  Total Marks: 253
```

Result:

Python fundamentals and the Student Record ADT were successfully built using simple procedural classes.

Ex. No: 02 Date:	Implementation of List ADT using Python Arrays

Aim:

Write a program to implement a list ADT using array in Python.

Algorithm:

1. Define the **ArrayList** class with an empty array as the initial state.
2. Implement methods for adding (**append**), inserting (**insert**), removing (**remove** and **remove_at**), and retrieving (**get**) elements from the array. Validate indices and handle errors appropriately.
3. Implement methods for checking if the array is empty (**is_empty**) and getting the length of the array (**len**).
4. Override the **str** method to provide a string representation of the array.
5. Test the functionality of the **ArrayList** class by creating an instance, performing operations (e.g., appending, inserting, removing), and printing the array.

Program:

```
class ArrayList:
    def __init__(self):
        self.array = []

    def len(self):
        return len(self.array)

    def is_empty(self):
        return len(self.array) == 0

    def get(self, index):
        if index < 0 or index >= len(self.array):
            raise IndexError("Index out of range")
        return self.array[index]

    def append(self, item):
        self.array.append(item)

    def insert(self, index, item):
        if index < 0 or index > len(self.array):
            raise IndexError("Index out of range")
        self.array.insert(index, item)

    def remove(self, item):
        try:
            self.array.remove(item)
```

```
            except ValueError:
                raise ValueError("Item not found")

        def remove_at(self, index):
            if index < 0 or index >= len(self.array):
                raise IndexError("Index out of range")
            del self.array[index]

        def __str__(self):
            return str(self.array)


    # Example usage
    my_list = ArrayList()
    print(my_list.is_empty())

    my_list.append(10)
    my_list.append(20)
    my_list.append(30)
    print(my_list)

    my_list.insert(1, 15)
    print(my_list)  #

    my_list.remove(20)
    print(my_list)

    my_list.remove_at(0)
    print(my_list)

    print(my_list.get(1))
```

Output:

```
True
[10, 20, 30]
[10, 15, 20, 30]
[10, 15, 30]
[15, 30]
30
```

Result:

The Python program for list ADT using array was executed successfully.

Ex. No: 03 Date:	Implementation of Singly, Doubly, and Circular Linked Lists

Aim:

To implement Singly Linked List, Doubly Linked List, and Circular Linked List in Python, and to perform insertion and traversal (display) operations on each

Algorithm:

1. Start the program.
2. Define a Node class with data and next pointer for the singly/circular list, and a DNode class with data , prev,and next pointers for the doubly linked list.
3. Singly Linked List: insert() traverses to the last node (if any) and links the new node to it; display() starts at the head and prints each node's data until the next pointer is None.
4. Doubly Linked List: insert() links the new node's prev pointer to the current last node and updates the last node's next pointer; display() traverses through the next pointers from the head.
5. Circular Linked List: insert() links the new node at the end and points it back to the head so the list forms a circle; display() traverses the list and stops when it comes back to the head.
6. Create objects of each list type, insert a few elements into each, and display the resulting list.
7. Stop the program.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class DNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


# ---------- Singly Linked List ----------
class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
```

```
        node = Node(data)
        if self.head is None:
            self.head = node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = node

    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")


# ---------- Doubly Linked List ----------
class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        node = DNode(data)
        if self.head is None:
            self.head = node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = node
            node.prev = temp

    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")


# ---------- Circular Linked List ----------
class CircularLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        node = Node(data)
        if self.head is None:
```

```
            self.head = node
            node.next = self.head
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            temp.next = node
            node.next = self.head

    def display(self):
        if self.head is None:
            print("Empty")
            return
        temp = self.head
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.head:
                break
        print("(back to head)")


if __name__ == "__main__":
    print("----- Singly Linked List -----")
    sll = SinglyLinkedList()
    sll.insert(10)
    sll.insert(20)
    sll.insert(30)
    sll.display()

    print("\n----- Doubly Linked List -----")
    dll = DoublyLinkedList()
    dll.insert(1)
    dll.insert(2)
    dll.insert(3)
    dll.display()

    print("\n----- Circular Linked List -----")
    cll = CircularLinkedList()
    cll.insert(100)
    cll.insert(200)
    cll.insert(300)
    cll.display()
```

OUTPUT:

```
----- Singly Linked List -----
10 -> 20 -> 30 -> None

----- Doubly Linked List -----
1 <-> 2 <-> 3 <-> None

----- Circular Linked List -----
100 -> 200 -> 300 -> (back to head)
```

Result:

The Python programs to implement Singly, Doubly, and Circular Linked Lists were executed successfully, and the insertion and display operations were verified.

Ex. No: 04 Date:	Implementation of Stack ADT and its Applications

Aim:

To implement the Stack Abstract Data Type (ADT) using a Python list and to apply it to check whether an expression has balanced parentheses.

Algorithm:

1. Start the program.
2. Create an empty list to represent the stack.
3. push(item): Append the item to the end of the list (top of the stack).
4. pop(): Remove and return the last element of the list; report Stack Underflow if the stack is empty.
5. peek(): Return the last element of the list without removing it.
6. Balanced Parentheses: Scan the expression from left to right. Push every opening bracket '(', '{', '[' onto the stack. On encountering a closing bracket, pop the stack and check whether it matches the corresponding opening bracket. The expression is balanced if the stack is empty after scanning the entire expression.
7. Test the stack operations and the balanced-parentheses application with sample expressions, and display the results.
8. Stop the program.

Program:

```
stack = []

# Push operation
def push(item):
    stack.append(item)
    print(f"Pushed {item}")

# Pop operation
def pop():
    if not stack:
        print("Stack Underflow")
        return None
    return stack.pop()

# Peek operation
def peek():
    if not stack:
        print("Stack is empty")
        return None
    return stack[-1]
```

```
# Application: Balanced Parentheses Checking
def is_balanced(expr):
    s = []
    for ch in expr:
        if ch in "({[":
            s.append(ch)
        elif ch in ")}]":
            if not s:
                return False
            top = s.pop()
            if (ch == ")" and top != "(") or \
               (ch == "}" and top != "{") or \
               (ch == "]" and top != "["):
                return False
    return len(s) == 0


if __name__ == "__main__":
    print("----- Basic Stack Operations -----")
    push(10)
    push(20)
    push(30)
    print("Stack:", stack)
    print("Peek:", peek())
    print("Popped:", pop())
    print("Stack after pop:", stack)

    print("\n----- Application: Balanced Parentheses -----")
    expressions = ["{[()]}", "([)]", "((a+b))"]
    for e in expressions:
        result = "Balanced" if is_balanced(e) else "Not Balanced"
        print(f"{e} -> {result}")
```

OUTPUT:

```
 ----- Basic Stack Operations -----
Pushed 10
Pushed 20
Pushed 30
Stack: [10, 20, 30]
Peek: 30
Popped: 30
Stack after pop: [10, 20]

----- Application: Balanced Parentheses -----
{[()]} -> Balanced
([)] -> Not Balanced

((a+b)) -> Balanced
```

Result:

The Python program to implement the Stack ADT and its application to check balanced parentheses was executed successfully, and the output was verified.

Ex. No: 05 Date:	Implementation of Queue ADT and its Applications

Aim:

To implement the Queue Abstract Data Type (ADT) using a Python list and to apply it to simulate a First-Come-First-Served (FCFS) printer queue

Algorithm:

1. Start the program
2. Create an empty list to represent the queue.
3. enqueue(item): Append the item to the end of the list (rear of the queue).
4. dequeue(): Remove and return the first element of the list (front of the queue); report Queue
5. Underflow if the queue is empty.
6. Printer Queue Application: Store the list of print jobs in a queue. Repeatedly remove the job at the front of the queue and process (print) it, until the queue becomes empty.
7. Test the basic queue operations and the printer-queue application, displaying results after each operation
8. Stop the program.

Program:

```
queue = []

# Enqueue operation
def enqueue(item):
    queue.append(item)
    print(f"Enqueued {item}")

# Dequeue operation
def dequeue():
    if not queue:
        print("Queue Underflow")
        return None
    return queue.pop(0)

# Application: Simulating a Printer Queue (First Come First Serve)
def process_print_jobs(jobs):
    q = list(jobs)
    print("Processing print jobs in order:")
    while q:
        job = q.pop(0)
```

```
        print(f"Printing job: {job}")

if __name__ == "__main__":
    print("----- Basic Queue Operations -----")
    enqueue(10)
    enqueue(20)
    enqueue(30)
    print("Queue:", queue)
    print("Dequeued:", dequeue())
    print("Queue after dequeue:", queue)

    print("\n----- Application: Printer Queue Simulation -----")
    jobs = ["Doc1.pdf", "Doc2.pdf", "Doc3.pdf"]
    process_print_jobs(jobs)
```

Output:

```
----- Basic Queue Operations -----
Enqueued 10
Enqueued 20
Enqueued 30
Queue: [10, 20, 30]
Dequeued: 10
Queue after dequeue: [20, 30]

----- Application: Printer Queue Simulation -----
Processing print jobs in order:
Printing job: Doc1.pdf
Printing job: Doc2.pdf
Printing job: Doc3.pdf
```

Result:

The Python program to implement the Queue ADT and its application to simulate a printer queue was executed successfully, and the output was verified.

Ex. No: 06 Date:	Implementation of Sorting and Searching Algorithms

Aim:

To implement Linear Search, Binary Search, and the sorting algorithms Bubble Sort, Selection Sort, and Insertion Sort ,quick sort and Merge sort using Python.

Algorithm:

1. Start the program.
2. Linear Search: Traverse the array sequentially and compare each element with the key; return the index on a match, else return -1.
3. Binary Search: On a sorted array, repeatedly compare the key with the middle element and narrow the search to the left or right half until found or the search space is empty.
4. Bubble Sort: Repeatedly compare adjacent elements and swap them if they are in the wrong order, so the largest element 'bubbles' to the end in each pass.
5. Selection Sort: In each pass, find the minimum element from the unsorted part of the array and swap it with the first unsorted element.
6. Insertion Sort: Take each element and insert it into its correct position within the already-sorted part of the array.
7. Quick Sort: Choose a pivot element from the array; partition the remaining elements into two sublists — those less than the pivot and those greater than the pivot — then recursively apply the same process to both sublists and combine them with the pivot in between.
8. Merge Sort: Divide the array into two halves; recursively sort each half; then merge the two sorted halves back into a single sorted array by comparing elements one by one.
9. Apply all algorithms on a sample array and display the results.
10. Stop the program..

Program:

```
# ---------- SEARCHING ----------

def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

def binary_search(arr, key):
```

```
    arr.sort()
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

# ---------- SORTING ----------

def bubble_sort(arr):
    a = arr.copy()
    n = len(a)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
    return a

def selection_sort(arr):
    a = arr.copy()
    n = len(a)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if a[j] < a[min_idx]:
                min_idx = j
        a[i], a[min_idx] = a[min_idx], a[i]
    return a

def insertion_sort(arr):
    a = arr.copy()
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
    return a

def quick_sort(arr):
    if len(arr) <= 1:
```

```
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + mid + quick_sort(right)

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# ---------- MAIN ----------

if __name__ == "__main__":
    arr = [64, 25, 12, 22, 11]
    key = 22

    print("Original Array:", arr)

    print("\n--- SEARCHING ---")
    print("Linear Search: Element found at index", linear_search(arr, key))
    print("Binary Search: Element found at index", binary_search(arr.copy(), key))

    print("\n--- SORTING ---")
    print("Bubble Sort   :", bubble_sort(arr))
    print("Selection Sort:", selection_sort(arr))
    print("Insertion Sort:", insertion_sort(arr))
    print("Quick Sort    :", quick_sort(arr))
    print("Merge Sort    :", merge_sort(arr))
```

Output

.

```
Original Array: [64, 25, 12, 22, 11]

--- SEARCHING ---
Linear Search: Element found at index 3
Binary Search: Element found at index 3

--- SORTING ---
Bubble Sort   : [11, 12, 22, 25, 64]
Selection Sort: [11, 12, 22, 25, 64]
Insertion Sort: [11, 12, 22, 25, 64]
Quick Sort    : [11, 12, 22, 25, 64]
Merge Sort    : [11, 12, 22, 25, 64]
```

Result:

The Python programs implementing Linear Search, Binary Search, and sorting algorithms (Bubble, Selection, and Insertion Sort,Quick and,Merge Sort) were executed successfully, and the outputs were verified.

Ex. No: 7 Date:	Implementation of Hash Tables and Collision Handling Technique

Aim:

To implement a Hash Table in Python using an array of buckets and to handle collisions using the Separate Chaining technique.

Algorithm:

1. Start the program.
2. Create a hash table as a list of empty lists (buckets), with a fixed table size.
3. Define a hash function hash_function(key) = key % table_size.
4. insert(key, value): Compute the bucket index using the hash function and append the(key,value) pair to that bucket's list, so that multiple keys can coexist in the same bucket(chaining).
5. search(key): Compute the bucket index and linearly scan the corresponding list to find and return the value associated with the key.
6. display(): Print the contents of every bucket in the hash table.
7. Insert a sample set of keys, display the table to observe collisions being chained, and perform a search operation.
8. Stop the program

Program:

```
TABLE_SIZE = 5
hash_table = [[] for _ in range(TABLE_SIZE)]


def hash_function(key):
    return key % TABLE_SIZE


def insert(key, value):
    index = hash_function(key)
    hash_table[index].append((key, value))
    print(f"Inserted key {key} at bucket {index}")


def search(key):
    index = hash_function(key)
    for pair in hash_table[index]:
        if pair[0] == key:
            return pair[1]
```

```
    return None

def display():
    for i in range(TABLE_SIZE):
        print(f"Bucket {i}: {hash_table[i]}")

if __name__ == "__main__":
    keys = [10, 15, 20, 3, 8]
    for k in keys:
        insert(k, f"value{k}")

    print("\nHash Table:")
    display()

    print("\nSearch key 15 ->", search(15))
    print("Search key 100 ->", search(100))
```

Output:

```
Inserted key 10 at bucket 0
Inserted key 15 at bucket 0
Inserted key 20 at bucket 0
Inserted key 3 at bucket 3
Inserted key 8 at bucket 3

Hash Table:
Bucket 0: [(10, 'value10'), (15, 'value15'), (20, 'value20')]
Bucket 1: []
Bucket 2: []
Bucket 3: [(3, 'value3'), (8, 'value8')]
Bucket 4: []

Search key 15 -> value15
Search key 100 -> None
```

Result:

The Python program implementing a Hash Table with the Separate Chaining collision-handling technique was executed successfully, and the output was verified.

Ex. No: 8 Date:	Implementation of Tree Traversal and Binary Search Tree Operations

Aim:

To implement a Binary Search Tree (BST) in Python and to perform insertion, searching, and the Inorder, Preorder, and Postorder traversal operations.

Algorithm:

1. Start the program.
2. Define a Node class with key, left, and right attributes.
3. insert(root, key): Starting from the root, recursively move left if the key is smaller or right if the key is larger than the current node's key, until a null position is found, and place the new node there.
4. search(root, key): Recursively compare the key with the current node; move left or right accordingly, and return True if found, else False.
5. Inorder Traversal: Recursively visit the left subtree, then the current node, then the right subtree (produces sorted order for a BST).
6. Preorder Traversal: Recursively visit the current node, then the left subtree, then the right subtree.
7. Postorder Traversal: Recursively visit the left subtree, then the right subtree, then the current node.
8. Insert a sample set of keys, display all three traversals, and perform search operations for a present and an absent key.
9. Stop the program.

Program:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root

def search(root, key):
    if root is None:
```

```
        return False
    if root.key == key:
        return True
    if key < root.key:
        return search(root.left, key)
    return search(root.right, key)

def inorder(root, result):
    if root:
        inorder(root.left, result)
        result.append(root.key)
        inorder(root.right, result)

def preorder(root, result):
    if root:
        result.append(root.key)
        preorder(root.left, result)
        preorder(root.right, result)

def postorder(root, result):
    if root:
        postorder(root.left, result)
        postorder(root.right, result)
        result.append(root.key)

if __name__ == "__main__":
    root = None
    values = [50, 30, 70, 20, 40, 60, 80]
    for v in values:
        root = insert(root, v)

    print("Inserted values:", values)

    inorder_result = []
    inorder(root, inorder_result)
    print("Inorder Traversal  :", inorder_result)

    preorder_result = []
    preorder(root, preorder_result)
    print("Preorder Traversal :", preorder_result)

    postorder_result = []
    postorder(root, postorder_result)
    print("Postorder Traversal:", postorder_result)

    print("\nSearch 60 ->", search(root, 60))
    print("Search 100 ->", search(root, 100))
```

Output:

```
Inserted values: [50, 30, 70, 20, 40, 60, 80]
Inorder Traversal  : [20, 30, 40, 50, 60, 70, 80]
Preorder Traversal : [50, 30, 20, 40, 70, 60, 80]
Postorder Traversal: [20, 40, 30, 60, 80, 70, 50]

Search 60 -> True
Search 100 -> False
```

Result:

The Python program to implement Binary Search Tree operations and Inorder, Preorder, and Postorder traversals was executed successfully, and the output was verified.

Ex. No: 09 Date:	Implementation of Graph Representation and Traversal Algorithms

Aim:

To represent a graph using an adjacency list in Python and to implement the Breadth-First Search (BFS) and Depth-First Search (DFS) graph traversal algorithms.

Algorithm:

1. Start the program.
2. Represent the graph using an adjacency list implemented with a Python dictionary, where each key is a vertex and its value is the list of adjacent vertices.
3. BFS (start): Initialize a queue with the start vertex and mark it visited. Repeatedly remove a vertex from the front of the queue, record it, and add all its unvisited neighbours to the queue, marking them visited, until the queue is empty.
4. DFS (start): Starting from the start vertex, mark it visited and record it, then recursively visit each unvisited neighbour, backtracking when no unvisited neighbours remain.
5. Display the graph's adjacency list, and perform BFS and DFS starting from a given vertex.
6. Stop the program

Program:

```
graph = {
   0: [1, 2],
   1: [0, 3],
   2: [0, 4],
   3: [1],
   4: [2]
}

def bfs(start):
   visited = [start]
   queue = [start]
   order = []
   while queue:
      node = queue.pop(0)
      order.append(node)
      for neighbor in graph[node]:
         if neighbor not in visited:
            visited.append(neighbor)
            queue.append(neighbor)
   return order

def dfs(start, visited=None, order=None):
   if visited is None:
      visited = []
      order = []
```

```
    visited.append(start)
    order.append(start)
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(neighbor, visited, order)
    return order

if __name__ == "__main__":
    print("Graph (Adjacency List):")
    for node in graph:
        print(node, "->", graph[node])

    print("\nBFS starting from node 0:", bfs(0))
    print("DFS starting from node 0:", dfs(0))
```

Output:

```
Graph (Adjacency List):
0 -> [1, 2]
1 -> [0, 3]
2 -> [0, 4]
3 -> [1]
4 -> [2]

BFS starting from node 0: [0, 1, 2, 3, 4]
DFS starting from node 0: [0, 1, 3, 2, 4]
```

Result:

The Python program to represent a graph using an adjacency list and to perform BFS and DFS traversals was executed successfully, and the output was verified.

Ex. No: 10 Date:	Implementation of Shortest Path and Minimum Spanning Tree Algorithms

Aim:

To implement Dijkstra's algorithm for finding the shortest path from a source vertex, and Prim's algorithm for finding the Minimum Spanning Tree (MST), using an adjacency matrix in Python

Algorithm:

1. Start the program.
2. Represent the weighted graph using an adjacency matrix, where graph[u][v] stores the weight of the edge between vertices u and v, and 0 indicates no edge.
3. Dijkstra's Algorithm: Initialize the distance of the source vertex as 0 and all others as infinity. Repeatedly select the unvisited vertex with the smallest known distance, mark it visited, and relax (update) the distances of its adjacent vertices if a shorter path is found through it.
4. Repeat the selection and relaxation process until all vertices are visited, producing the shortest distance from the source to every other vertex.
5. Prim's Algorithm: Start with an arbitrary vertex included in the MST. Repeatedly find the minimum-weight edge that connects a vertex already in the MST to a vertex not yet included, and add that edge and vertex to the MST.
6. Continue until all vertices are included in the MST, and compute the total cost as the sum of the selected edge weights.
7. Display the shortest distances from the source and the edges and total cost of the MST.
8. Stop the program.

Program:

```
INF = float('inf')

# Graph represented as an adjacency matrix (0 means no edge, except diagonal)
graph = [
    [0, 4, 4, 0, 0],
    [4, 0, 2, 5, 0],
    [4, 2, 0, 3, 0],
    [0, 5, 3, 0, 3],
    [0, 0, 0, 3, 0]
]
V = len(graph)


# ---------- Dijkstra's Shortest Path Algorithm ----------
def dijkstra(src):
    dist = [INF] * V
    dist[src] = 0
```

```
    visited = [False] * V

    for _ in range(V):
        u = -1
        for i in range(V):
            if not visited[i] and (u == -1 or dist[i] < dist[u]):
                u = i
        visited[u] = True

        for v in range(V):
            if graph[u][v] != 0 and not visited[v]:
                if dist[u] + graph[u][v] < dist[v]:
                    dist[v] = dist[u] + graph[u][v]
    return dist


# ---------- Prim's Minimum Spanning Tree Algorithm ----------
def prim():
    selected = [False] * V
    selected[0] = True
    edges = []
    total_cost = 0

    for _ in range(V - 1):
        min_edge = (None, None, INF)
        for u in range(V):
            if selected[u]:
                for v in range(V):
                    if not selected[v] and graph[u][v] != 0:
                        if graph[u][v] < min_edge[2]:
                            min_edge = (u, v, graph[u][v])
        u, v, w = min_edge
        selected[v] = True
        edges.append((u, v, w))
        total_cost += w
    return edges, total_cost


if __name__ == "__main__":
    print("----- Dijkstra's Shortest Path Algorithm (source = 0) -----")
    distances = dijkstra(0)
    for i, d in enumerate(distances):
        print(f"Shortest distance from node 0 to node {i} = {d}")

    print("\n----- Prim's Minimum Spanning Tree Algorithm -----")
    mst_edges, cost = prim()
    print("Edges in MST (u, v, weight):")
    for e in mst_edges:
        print(e)
    print("Total cost of MST =", cost)
```

Output:

```
----- Dijkstra's Shortest Path Algorithm (source = 0) -----
Shortest distance from node 0 to node 0 = 0
Shortest distance from node 0 to node 1 = 4
Shortest distance from node 0 to node 2 = 4
Shortest distance from node 0 to node 3 = 7
Shortest distance from node 0 to node 4 = 10

----- Prim's Minimum Spanning Tree Algorithm -----
Edges in MST (u, v, weight):
(0, 1, 4)
(1, 2, 2)
(2, 3, 3)
(3, 4, 3)
    Total cost of MST = 12
```

Result:

The Python programs implementing Dijkstra's Shortest Path algorithm and Prim's Minimum Spanning Tree algorithm using an adjacency matrix were executed successfully, and the outputs were verified.